

SECURE DIGITAL BANKING PLATFORM

Milestone 3 - Project Report

Core Focus: Payment Initiation, Beneficiary Verification, Fraud Check /Team c

1. Introduction

Secure is a digital banking platform that helps a bank handle customer accounts, money transfers, and transaction records in one connected system. Milestone 3 builds on this with a Payment Initiation suite: a guided flow that lets a customer send money to a saved beneficiary, with an automatic fraud/risk check running before the payment is completed.

The platform continues to use an Angular frontend, a Java Spring Boot backend, and a database layer (MySQL and PostgreSQL). This report gives a simple overview of the three modules delivered in this milestone — Payment Initiation, Beneficiary Verification, and Fraud Check — across the frontend, backend, and database.

Key Features

- Payment Initiation – send money to a saved beneficiary using IMPS, NEFT, RTGS, or UPI.
- Beneficiary Verification – add and verify people/accounts a customer is allowed to pay.
- Fraud Check – every payment is automatically scored for risk before it is approved.

2. Technology Stack

The project uses a standard 3-layer setup: a web frontend, a Java backend, and a database. Below is a simple summary of the tools used.

Layer	Technology	Purpose
Frontend	Angular 19	Builds the web pages the user interacts with
UI Library	Angular Material + SCSS	Provides ready-made forms, tables, buttons, and a consistent look
Backend	Java 17 + Spring Boot	Handles business logic and REST APIs
Data Access	Spring Data JPA / Hibernate	Connects the backend code to the database
Database	PostgreSQL / MySQL	Stores beneficiaries, payments, and fraud check records
Alternate DB Layer	PHP + MySQL	A separate, simpler implementation of the same 3 modules
Version Control	Git & GitHub	Tracks code changes and team collaboration

3. Frontend

The frontend is built using Angular 19 with standalone components. Each of the three modules is a separate routed screen, wrapped in a shared header/sidebar layout.

Module 1: Payment Initiation

This is the main screen where a customer creates a payment, guided through four steps: Form → Review → Processing → Success.

- Pick a 'from' account from a dropdown of the customer's own accounts.
- Choose a beneficiary — only 'Verified' beneficiaries can be selected; the 'to account' field fills in automatically.
- Enter the amount (must be greater than ₹0.00), pick a payment type (Transfer, Bill Payment, Other) and mode (IMPS, NEFT, RTGS, UPI), and add an optional note.
- Review the payment on a summary screen, then confirm — a short animated loader shows the risk check running.
- See a success screen with a generated transaction reference number and the payment's risk score.

1.1 Workflow

From Account + Verified Beneficiary + Amount + Type + Mode → Review → Confirm
→ Generate Transaction Reference → Status: Processing → Awaiting Fraud Check

1.2 Example

Request: From XXXXXX1234 → Beneficiary (Verified) → ₹5,000 → Transfer → UPI
Result: Payment saved as Processing, Transaction Ref TXN-20260825-4F91C2
Next step: Fraud Check must run before the payment can become Success.

The screenshot displays the 'Payment Initiation' screen within the 'FinCore Nexus CORE BANKING SYSTEM'. The interface is dark-themed and includes a sidebar with navigation options: 'Payment Initiation' (selected), 'Beneficiary Verification', and 'Fraud Check'. The main content area is titled 'Payment Initiation' with a subtitle 'Initiate secure interbank & intra-bank payments with real-time risk validation'. A status indicator shows 'Fraud Engine: Active'. The primary action is 'Initiate New Payment', which prompts the user to 'Enter payment parameters and select a verified beneficiary.' The form contains several fields: 'From Account Number' (set to 'XXXXXX1234 - Primary Savings (Bal: ₹2,45,000.00)'), 'Beneficiary' (set to 'John Mathew - XXXX1234 (HDFC Bank)'), 'To Account Number' (set to 'XXXX1234'), 'Amount' (set to '₹ 400000'), 'Payment Type' (set to 'Transfer'), and 'Payment Mode' (set to 'IMPS'). A 'Verified' status is shown next to the IFSC code 'HDFC0001234'. A text area for 'Description / Payment Remarks (Optional)' is present with the placeholder 'Enter transfer reference or remarks (e.g. Vendor Invoice Payment)'. A 'Review Payment' button is located at the bottom right of the form.

FinCore Nexus

CORE BANKING SYSTEM

Payment Initiation

AD Alex Developer
Payment Operator

MAIN NAVIGATION

Payment Initiation

Beneficiary Verification

Fraud Check

FinCore Platform
v2.4.0 • Payment Module

Payment Initiation

Initiate secure interbank & intra-bank payments with real-time risk validation

Fraud Engine: Active

Payment Review

Verify details carefully before confirming payment authorization.

FROM ACCOUNT

XXXXXX1234

BENEFICIARY

John Mathew

AMOUNT

₹400,000.00

PAYMENT MODE

IMPS

TO ACCOUNT

XXXXX1234

BANK NAME

HDFC Bank (IFSC: HDFC0001234)

PAYMENT TYPE

Transfer

DESCRIPTION

None provided

This payment will undergo automated FinCore Nexus real-time fraud checking upon confirmation.

Back / Edit

Confirm Payment

FinCore Nexus

CORE BANKING SYSTEM

Payment Initiation

AD Alex Developer
Payment Operator

MAIN NAVIGATION

Payment Initiation

Beneficiary Verification

Fraud Check

FinCore Platform
v2.4.0 • Payment Module

Payment Initiation

Initiate secure interbank & intra-bank payments with real-time risk validation

Fraud Engine: Active

Payment Initiated Successfully

Transaction processed and recorded in FinCore Core Banking Ledger

Payment Amount

₹400,000.00

Transaction Reference

TXN-20260825-SKTNMZ

From Account

XXXXXXXX1234

To Account

XXXXX1234

Beneficiary

John Mathew

Payment Mode

IMPS

Payment Type

Transfer

Fraud Screening Status

Safe (Risk Score: 6/100)

Initiated At

2026-08-25 07:43:44

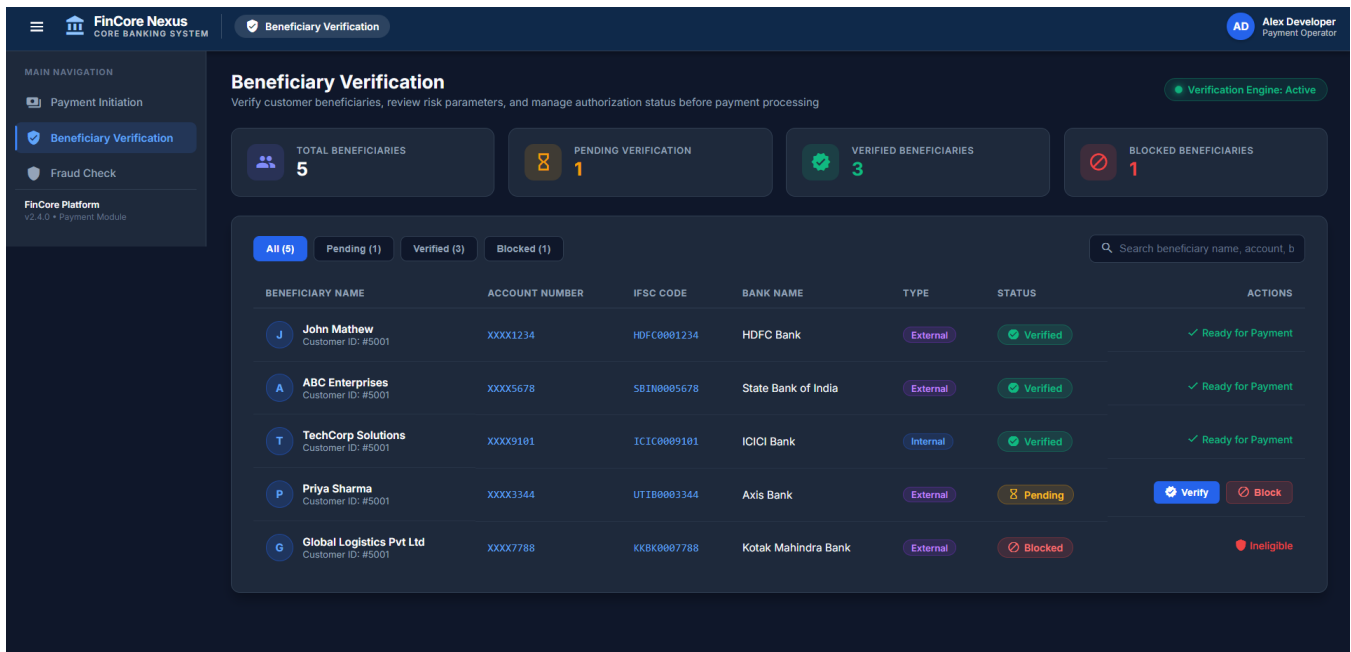
Done

Make Another Payment

Module 2: Beneficiary Verification

This screen lists all beneficiaries a customer has added, and lets an operator manage their status.

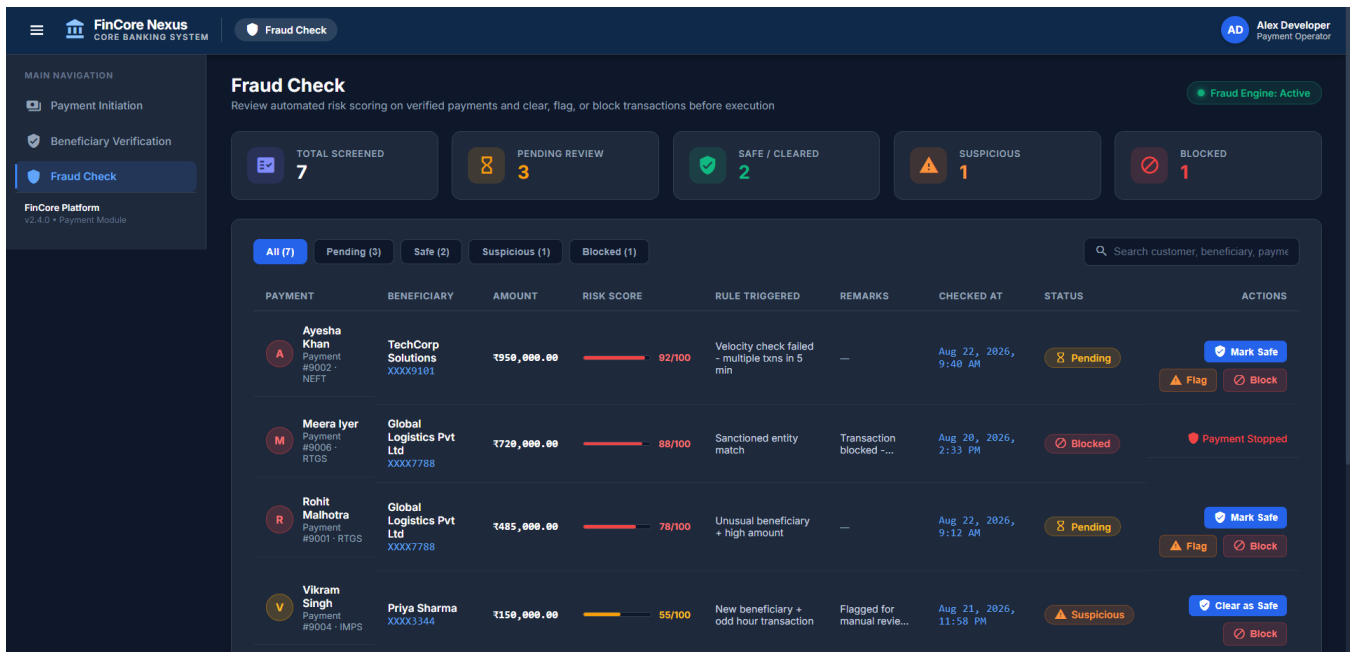
- Shows each beneficiary's name, account details, bank, and current status (Pending, Verified, or Blocked).
- Lets an operator verify a 'Pending' beneficiary, making it eligible to receive payments.
- Lets an operator block a beneficiary, preventing any future payments to it.



Module 3: Fraud Check

This screen shows the risk/fraud results for payments, for an operator to review.

- Lists fraud check records with a risk score, status (Pending, Safe, Suspicious, Blocked), and the rule that was triggered.
- Lets an operator mark a transaction Safe, flag it Suspicious for further review, or Block it outright.
- Each action updates the record with a new timestamp and an optional remark.



Shared Frontend Structure

- A shared header and sidebar give the app a consistent dashboard layout across all three modules.

- Each module has its own set of models (Payment, Beneficiary, Fraud Check) and a service file that supplies its data.

4. Backend

The backend is built using Java and Spring Boot. It exposes REST APIs for each of the three modules, following a clean layered structure: Controllers receive requests, Services contain the business logic, and Repositories talk to the database.

Module 1: Payment Initiation

The Payment module lets a payment be created, looked up, and listed.

- POST /api/payments – creates a payment. The status is always set to 'Processing' first, since the fraud check must run before a payment can be marked successful.
- A unique transaction reference is generated automatically, in the format TXN-YYYYMMDD-XXXXXX.
- GET /api/payments/{paymentId}, GET /api/payments, and GET /api/payments/transaction/{transactionRef} – fetch one payment, all payments, or a payment by its reference number.

Module 2: Beneficiary Verification

The Beneficiary module lets beneficiaries be listed, looked up, and updated.

- GET /beneficiary-verification/getList – returns all beneficiaries.
- GET /beneficiary-verification/getById/{id} – returns one beneficiary's details.
- PUT /beneficiary-verification/updateDetails – updates a beneficiary's name, account, bank, IFSC, type, or status (e.g. moving it from Pending to Verified).

Module 3: Fraud Check

The Fraud Check module scores a payment and updates its status based on the result.

- POST /fraud-check – records a risk score and fraud status for a payment, then automatically updates that payment's status to match.

```
Safe -> Payment marked Success
Suspicious -> Payment stays Processing (held for review)
Blocked -> Payment marked Failed
Pending -> Payment stays Processing
```

- GET /fraud-check/payment/{paymentId} – fetches the fraud check result for a specific payment.

5. Database

The database stores the data used by all three modules: beneficiaries, payments, and fraud check results. The active application uses PostgreSQL through Spring Data JPA/Hibernate. The project also ships a separate MySQL schema with matching PHP scripts, useful for testing outside the Java backend.

5.1 Data Design

Three main tables make up the schema for this milestone, each linked back to a parent record by a foreign key.

Table	Key Columns	Notes
beneficiary	beneficiary_id (PK), customer_id (FK), beneficiary_name, account_no, ifsc_code, bank_name, beneficiary_type, status, created_at	beneficiary_type: Internal/External; status: Pending/Verified/Blocked
payment	payment_id (PK), from_account_no (FK), to_account_no (FK), beneficiary_id (FK), amount, payment_type, payment_mode, payment_status, transaction_ref (unique), description, initiated_at, updated_at	payment_mode: IMPS/NEFT/RTGS/UPI; payment_status: Pending/Processing/Success/Failed/Cancelled
fraud_check	fraud_check_id (PK), payment_id (FK, unique), risk_score, fraud_status, rule_triggered, remarks, checked_at	fraud_status: Pending/Safe/Suspicious/Blocked; one fraud check per payment

How the tables connect: a Beneficiary belongs to a Customer; a Payment links a 'from' account, a 'to' account, and a Beneficiary; and a Fraud Check links to exactly one Payment. This lets the database enforce that a payment can't exist without a valid beneficiary, and a fraud check can't exist without a valid payment.

Sample data included with the schema: two Verified beneficiaries (Amit Verma, Rahul Sharma), two sample payments (₹3,000 via IMPS and ₹5,000 via NEFT), and matching fraud check results — one scored 15 and marked 'Safe', the other scored 72 and marked 'Suspicious' — enough to test all three modules right away.

5.2 PHP Scripts

Alongside the SQL schema, the DB folder includes PHP scripts (using mysqli with prepared statements) that perform the same three-module operations directly against MySQL, independent of the Java backend.

Script	Module	What it does
beneficiary_verification.php	Beneficiary Verification	Validates the customer and account exist, then inserts a new beneficiary record
payment_initiation.php	Payment Initiation	Validates the payment details and amount, checks the sender account, then creates a payment
fraud_check.php	Fraud Check	Looks up a payment and its beneficiary, checks the beneficiary is Verified, then scores the payment for risk based on its amount

fraud_check.php works out the risk directly from the payment amount, using three simple bands: amounts of ₹1,00,000 or more are scored 80 and marked 'Suspicious'; amounts from ₹50,000 to ₹99,999 are scored 50 and marked 'Suspicious'; and anything below ₹50,000 is scored 15 and marked 'Safe'. This gives a quick, rule-based way to flag higher-value payments for review without needing the full Java backend running.

All three scripts use prepared statements with bound parameters, which is a safer way to build SQL queries and helps prevent SQL-injection issues.

6. Conclusion

This milestone successfully adds the Payment Initiation feature to the Secure platform, along with beneficiary verification and an automatic fraud check for every transfer. Each of the three modules is implemented consistently across the frontend, backend, and database — the Angular frontend gives users a simple, guided flow; the Spring Boot backend applies the right checks and hands off payment status based on the fraud check result; and the database (both the PostgreSQL schema and the parallel MySQL/PHP scripts) keeps beneficiaries, payments, and fraud checks properly connected.

Overall, the project demonstrates a clean, real-world style banking system where the frontend, backend, and database work together smoothly across all three modules — making the system easier to understand, test, and extend in future milestones.

7. Team C

Team Member	Role
Manikandan	Frontend Developer
Pavithra	Frontend Developer
Kousalya	Frontend Developer
Nithish	Backend Developer
Jeevana	Backend Developer
Raghvendra	Database Developer